5 Optimization for LLM inference

5.1 environment

5.1.1 Hardware environment

Item Name	Description	Quantity
Server	(INSPUR)NF8480M5	3
CPU	Intel Platinum 8352Y 3.90 GHz	10
Memory	P1-DIMMC2 DDR4 3200MHZ 32768MB	8
	P2-DIMMC2 DDR4 3200MHZ 32768MB	16
GPU	NVIDIA A100-SXM4-40GB	6
PSU	PWS-2K21G-2R	6
Switch	GbE switch	1
Raid	(INSPUR) AS2150G2 Raid 4*16GB FC / 6*8TB SAS	1
Cable	Infiniband cable	5
Disk	512GB	1
Disk	2TB	2

5.1.2 Software environment

To run the application, some external libraries are required. All these libraries, along with base code, can be accessed through the officially provided GitHub repository (https://github.com/ASC-Competition/ASC24-LLM-inference-optimization).

Moreover, We have used some modules provided by the supercomputing center, which can be implemented through the following commands.

```
1 module load intel/oneapi
2 module load nvidia/cuda/11.6
```

```
module load scl/gcc11.2
module load cmake/3
```

5.2 Test methods

5.2.1 Test node usage

By comparison, we can see that the efficiency of multi-node parallel test improvement is very obvious. Therefore, this is a feasible solution to optimize the performance by improving the hardware environment.

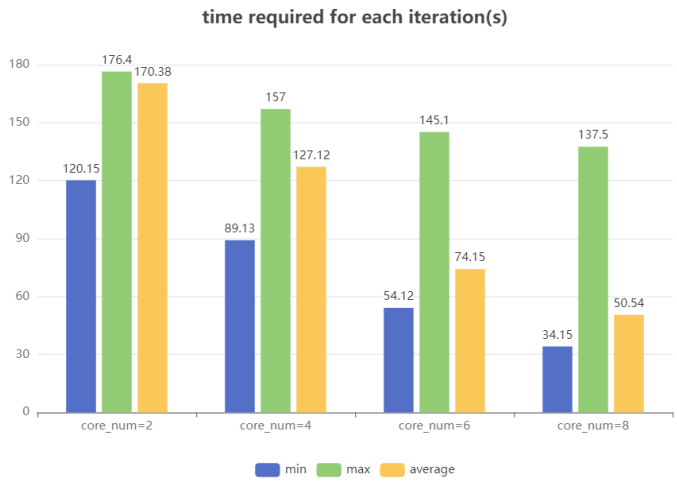


Figure. 5.1 Time required for each iteration for different GPU numbers

5.3 Performance Optimization and Performance Improvement Evaluation

5.3.1 GPU acceleration

```
async def main_async(args):
    print(args)
    random.seed(args.seed)
    if args.dataset is None:
        # Synthesize a prompt with the given input length.
        prompt = "hi" * (args.input_len - 1)
        requests = [(prompt, args.input_len, args.output_len)
                    for _ in range(args.num_samples)]
    else:
        with open(args.dataset) as f:
            requests = json.load(f)
    if args.num_samples is not None:
```

```
        requests = requests[0:args.num_samples]
    tasks = []
    tasks.append(collect_gpu_status(1, "gpu_status.json", args.jobID
        ))
    tasks.append(run_hf_async(requests, args.model, args.
        trust_remote_code, args))
    await asyncio.gather(*tasks)
```

We used the Nvidia Nsight System to analyze the reasons for program performance bottlenecks.We found that the baseline always uses the CPU while tokenizing, while the GPU keeps spinning, resulting in low utilization. So we handed over the tokenize part of the operation to the GPU, and only allowed the CPU to emit instructions. By properly scheduling CPU and GPU, we have significantly improved throughput.

5.3.2 Parameter settings

We used Vtune for program hotspot analysis, and the following are the analysis results. We found that the computational load is mainly concentrated in

```
tokenizer = LlamaTokenizer.from_pretrained(tokenizer_path)
llm = LlamaForCausalLM.from_pretrained(llm_path, device_map='auto',
    torch_dtype=torch.float16)
```

From the above analysis, we can see that the GPU has a delay when loading data each time, so it is not possible to achieve good parallelism when using the CPU to send data. Therefore, for the relatively large model inference of Llama2, we designed its underlying logic that supports multiple cards in parallel with the model.

The core issue of parallelism is how to segment models and perform segmentation operations, as KQV requires matrix multiplication with input X to obtain tensors.

Multi card parallelism is achieved at the device level, and the core logic is to split the computation into different devices. After the computation is completed, the results are synchronized to the next device (GPU or CPU) to complete the multi card model parallelism mechanism at each level. The logic of device splitting is simply to divide the matrix into blocks and maintain a consistent positional relationship with the parameter loading.

5.3.3 KV cache

KV cache is a typical method of improving inference performance through the idea of exchanging space for time.

In order to facilitate the understanding of the working principle of kV cache, we need to first understand that the process of LLM inference is an autoregressive process, and each time a token is generated, attention operations need to be combined with all previous tokens. That is to say, the first i-th token will be fed into the model as the prediction data for the i+1st time, in order to obtain the inference token for the i+1st time.

5.3.4 Performance Improvement evaluation

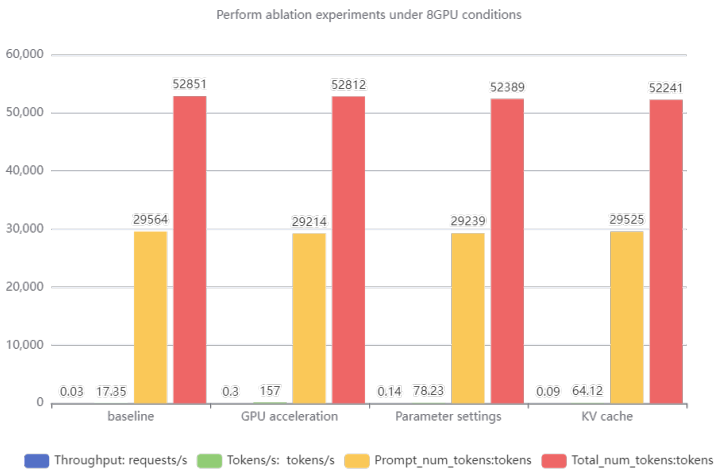


Figure. 5.2 Perform ablation experiments under 8GPU conditions

To summarize, we conducted ablation experiments under 8GPU conditions and found that using a mixture of CPU and GPU, parameter settings, and kV cache significantly improved the experimental performance indicators throughput and token/s.